EXPERIMENT 14

Exploring pro-code extension of a Zoho Creator application using Zoho Catalyst / custom functions.

Student details	Project details
HARIHARAN N Register No. 241401024	Insurance Claim Request & Processing Management
Department: CSBS	Platform: Zoho Creator (India region)
Institution: Rajalakshmi Engineering College	Evidence / verification date: 19 October 2026

Aim

To explore custom-function and pro-code extension patterns for insurance analytics, implement and test a backend example, and record the pending Catalyst deployment accurately.

Execution status

Creator custom-function exploration and a local Node.js backend example were completed. Sixteen local tests passed. Catalyst onboarding, cloud deployment, generated endpoint and live Creator-to-Catalyst API integration remain PENDING as requested.

Application context

The project records insurance requests, validates submitted details, assigns claims for processing and presents claim status and amount summaries. Existing forms, reports, Deluge workflows and permission profiles provide the base for this experiment.

Screenshots in this report are actual captures from the project account. Explanations, tables and code listings are documentation, not simulated platform screenshots.

Extension design and implementation

1. Used the existing Creator Deluge function as an application-side example of code extending declarative forms and reports.

2. Implemented a separate Node.js claim-summary module with a Catalyst Basic I/O adapter.

3. Accepted only claim_type, status and requested_amount; rejected unexpected fields and invalid inputs.

4. Executed local tests with mocked Basic I/O methods. Tested both aggregation logic and adapter error handling.

5. Opened Catalyst onboarding and stopped before accepting its terms. No cloud project, endpoint or authorized Catalyst connection was created.

Contract	Implementation
Input	payload JSON with a claims array; maximum 1,000 records.
Allowed fields	claim_type, status, requested_amount only; no contact, identity or medical fields.
Validation	Known type/status; positive finite numeric INR amount; maximum 1,000,000,000 per claim; at most two decimal places.
Output	Total/pending counts, requested INR, counts by type and status. Approved and Rejected are excluded from pending.
Arithmetic	Accumulate integer paise, then divide by 100.
Errors	Adapter returns status 400 and structured error; writes once and closes context.

Proposed integration boundary

Future flow: authorized Creator function -> authenticated Catalyst endpoint -> validated aggregate response -> Creator display. The endpoint and connection are not yet configured. Do not place credentials in Deluge source or send full claim records when only summary fields are needed.

Code and actual local test results

```
module.exports = async (context, basicIO) => {
  try {
    const raw = basicIO.getArgument('payload');
    const payload = typeof raw === 'string'
      ? JSON.parse(raw) : raw;
    basicIO.write(JSON.stringify({
      ok: true, summary: summarize(payload)
    }));
  } catch (error) {
    basicIO.setStatus(400);
    basicIO.write(JSON.stringify({ok: false, error:
      error instanceof SyntaxError ? 'Invalid JSON' : error.message}));
  } finally { context.close(); }
};
```

Test group	Actual local outcome
Aggregation	Three fixtures: INR 155,000, two pending; empty input zero; Rejected excluded - PASS.
Data validation	Missing array, zero, unknown status, extra email field and over-size batch rejected - PASS.
Number validation	NaN, Infinity, negative, numeric string and excess decimal precision rejected - PASS.
Precision	0.1 + 0.2 processed using integer paise - PASS.
Adapter	Valid input writes once and closes; malformed JSON returns 400 and closes - PASS.
Total	16 / 16 tests PASS in local Node.js; no cloud execution.

Local fixtures use Submitted, Under Review and Approved. They are separate from the Creator Development records, whose Travel record was later set to Rejected. Similar totals do not establish a live integration.

Evidence and pending Catalyst work

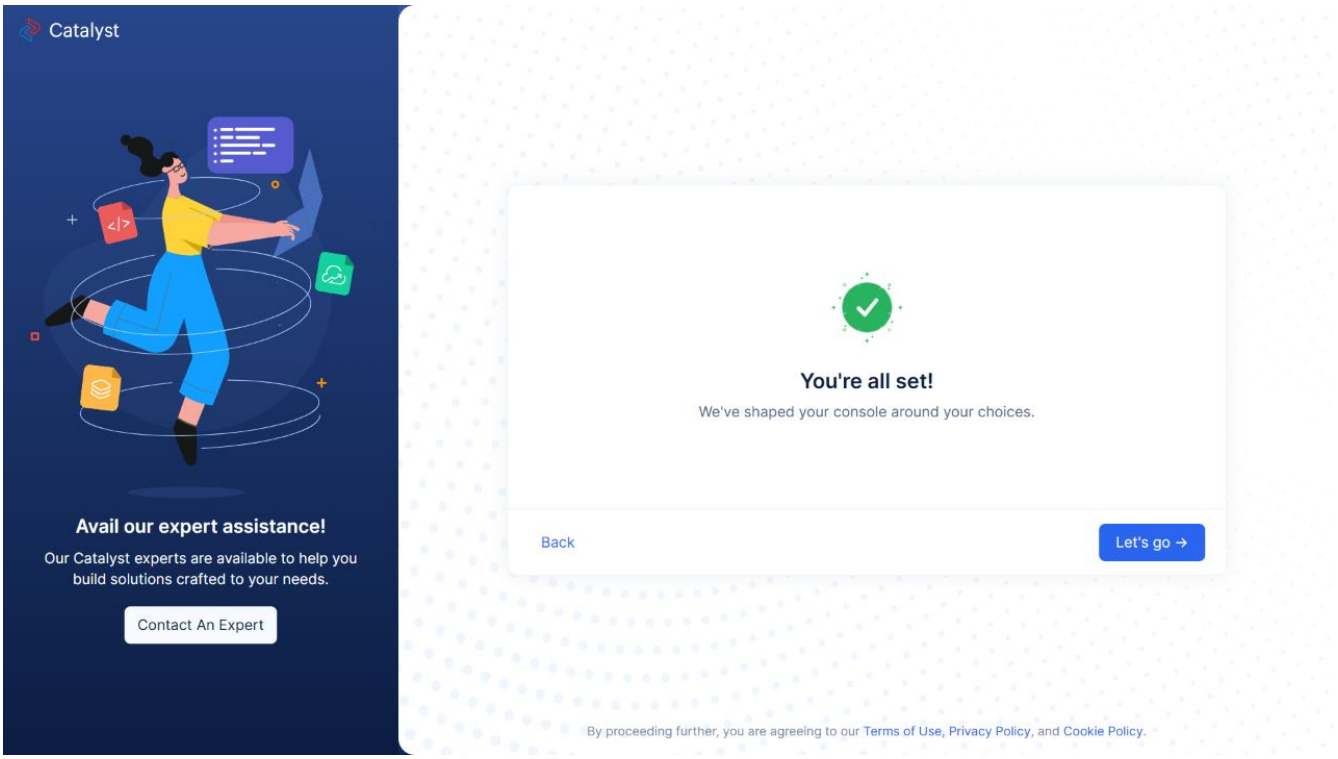


Figure 14.1 - Real Catalyst onboarding screen where setup remains pending.

Files prepared

The accompanying backend source consists of index.js, test.js, package.json, README.txt and test-results.json under outputs/backend/claim-summary. The code is locally tested, not a deployed cloud service.

Remaining steps

Complete authorized onboarding, create a supported Node.js function, deploy and test synthetic input in Catalyst, obtain the generated URL, configure authenticated access and a named Creator Connection, and verify live success/error responses. These actions have not been performed.

Result

Pro-code extension was explored through custom functions and a locally tested backend. Catalyst remains pending; no cloud integration success is claimed.

References

Catalyst Basic I/O functions: https://docs.catalyst.zoho.com/en/serverless/help/functions/basic-io/